

Bac à Sable Technologique et Structure du projet

1. Architecture Globale du Système

Le projet de chaque groupe sera un Système Multi-Agents (SMA) interconnecté et automatisé.

Le système doit intégrer les 4 couches suivantes :

1. Couche 1 : Orchestration

Utilisation de n8n pour modéliser visuellement le flux de travail (Workflow). n8n servira de bus de communication entre les agents et de gestionnaire d'événements (Triggers). Idéalement en hébergement local via Docker ou version Cloud gratuite.

2. Couche 2 : Cognitive

Un minimum de 3 Agents distincts configurés via les nœuds *Advanced AI* de n8n, avec des *System Prompts* spécifiques et une gestion stricte de la mémoire (Context Window).

3. Couche 3 : Connaissance

Une Base de Données Vectorielle (Vector DB) alimentée par des documents métiers, interrogeable de manière sémantique par au moins l'un des agents.

4. Couche 4 : Action et Résilience

Les agents ne doivent pas juste "communiquer", ils doivent interagir avec leur écosystème de manière autonome et sécurisée via :

- L'Interfaçage Cloud et l'utilisation d'outils bureautiques (Google Sheets, Gmail ou autres) pour la lecture/écriture de données et la communication sortante.
- Le Standard MCP (Model Context Protocol) et la mise en œuvre d'un client MCP pour unifier l'accès aux ressources critiques (SQL, API métier, fichiers locaux), garantissant une abstraction totale entre la logique de l'agent et la source de données.
- La capacité de l'agent à interpréter le retour d'une action (succès/échec) et à déclencher un chemin de repli en cas d'erreur technique via une boucle d'observation & Fallback.
- L'intégration de points de validation pour les actions à fort impact (Human-in-the-Loop), assurant une supervision humaine sur les sorties critiques de l'IA.

2. Le Processus d'ingénierie du projet – Development Workflow

Je vous propose un processus de développement itératif en 5 phases. Pour chaque étape, j'ai formulé des questions afin de vous aider à mieux comprendre les enjeux et à progresser dans votre projet. Considérez-les comme une check-list pour mener à terme votre projet.

Phase 1 : Modélisation conceptuelle (Design System)

- Comment pouvez-vous cartographier le flux de données de votre système ? A quoi ressemble ce flux sous forme de logigramme ?
- Quelle topologie de SMA avez-vous choisie pour votre système ? S'agit-il d'une architecture hiérarchique (avec un Agent Manager qui distribue les tâches), séquentielle (pipeline $A \rightarrow B \rightarrow C$), ou d'une autre organisation ?
- Pourquoi ce choix d'architecture est-il pertinent pour votre problème ?
- Quels agents composent votre système ? Pour chaque agent, quel est son rôle précis dans le flux ? Quels outils lui sont attribués ? Quel est l'objectif ou la tâche principale que cet agent doit accomplir ?

Phase 1 : Plus Plus :

- En observant votre logigramme, quel est le "Single Point of Failure" (point de rupture unique) de votre flux ? Comment l'avez-vous sécurisé pour éviter un blocage complet du SMA ?
- Si nous devons supprimer un seul de vos agents pour réduire les coûts, lequel serait-ce et comment redistribueriez-vous ses responsabilités et ses outils aux autres ?
- Quelles difficultés avez-vous rencontrées dans cette phase et comment les avez-vous surmontées ?

Phase 2 : Ingénierie de la Connaissance (Pipeline RAG)

- Pour votre projet, à quoi ressemble un premier workflow n8n dédié uniquement à l'ingestion des données ?
- Quelles sources de données allez-vous intégrer (PDF, Web, autres) ?
- Comment allez-vous structurer les étapes du pipeline d'ingestion : extraction des données, découpage (chunking), calcul des embeddings et stockage dans la base vectorielle ?
- Quelle stratégie de découpage avez-vous choisie pour vos PDF et pourquoi ?
- Comment avez-vous géré le risque de perte de contexte sémantique à la jonction entre deux chunks ?

Phase 2 : Plus Plus :

- Comment allez-vous vérifier que les données ont bien été indexées et sont exploitables par votre système ?
- Comment garantissez-vous que les embeddings générés sont réellement pertinents pour les questions spécifiques de vos utilisateurs ?
- Quelles difficultés avez-vous rencontrées dans cette phase et comment les avez-vous surmontées ?

Phase 3 : Prompt Engineering & Outillage (Unitaire)

- Comment pouvez-vous tester chaque agent individuellement avant de les intégrer dans le système global ?
- Comment allez-vous rédiger les System Prompts de vos agents en appliquant les techniques vues en cours ou à travers votre recherche ?
- Comment pouvez-vous connecter votre agent à des outils externes ?

Phase 3 : Plus Plus :

- Quels types de tests unitaires ou de "logs de raisonnement" utilisez-vous pour identifier une dérive (hallucination) entre la perception de la donnée et l'exécution finale ?
- Comment isolez-vous une défaillance dans ce flux : est-ce une erreur de lecture, une erreur d'interprétation logique, ou un échec d'exécution de l'outil ?
- Quelles difficultés avez-vous rencontrées dans cette phase et comment les avez-vous surmontées ?

Phase 4 : Orchestration et Protocole MCP

- Comment allez-vous relier les différents agents entre eux dans n8n afin de former un système multi-agents cohérent ?
- Comment l'information circule-t-elle entre les agents dans votre flux ?
- Comment gérez-vous la transmission de la mémoire ou du contexte d'un agent à l'autre sans dépasser la limite de Max Tokens ?
- Quelles stratégies avez-vous mises en place pour optimiser l'utilisation de la fenêtre de contexte (résumé, filtrage, mémoire externe, ...) ?
- Comment avez-vous intégré une connexion MCP dans votre système ?

Phase 4 : Plus Plus :

- En quoi l'adoption du standard MCP modifie-t-elle la sécurité et la modularité de votre architecture par rapport à une intégration directe (hardcoded) ?
- Quelles difficultés avez-vous rencontrées dans cette phase et comment les avez-vous surmontées ?

Phase 5 : Gestion des erreurs et Robustesse

- Quels mécanismes de gestion des erreurs avez-vous mis en place dans votre workflow ?
- Comment avez-vous implémenté des chemins de repli (fallback) pour garantir la robustesse du système ?
- Que se passe-t-il si l'API du LLM ne répond pas ?
- Comment le système réagit-il si l'agent ne trouve pas l'information dans la base RAG ?
- En cas d'hallucination ou un format JSON invalide, votre orchestrateur est-il doté d'une boucle d'auto-correction (Self-Correction loop) ou d'un validateur de schéma capable de forcer l'agent à re-générer une sortie conforme ?
- Quelles stratégies avez-vous mises en place pour assurer la fiabilité et la continuité du système malgré ces erreurs potentielles ?

Phase 5: Plus Plus:

- Quelles difficultés avez-vous rencontrées dans cette phase et comment les avez-vous surmontées ?
- Si votre système devait être utilisé dans un contexte réel (entreprise, service client, etc.), quelles autres situations d'erreur devriez-vous anticiper ?

3. Bonnes pratiques et soutenance technique :

- Pourquoi avoir utilisé trois agents plutôt qu'un seul grand prompt ? Quelles autres alternatives avez-vous envisagées et/ou testées ? (Ici, nous évaluons la pertinence du agentique décomposition ainsi que la capacité d'auto-évaluation).
- L'agent a-t-il réellement utilisé Google Sheets (ou un autre outil) de manière autonome grâce à son raisonnement, ou avez-vous dû forcer cette action dans le workflow ? (Nous nous intéressons ici à l'autonomie du tool usage vs orchestration et au débogage du système).
- Quelle stratégie de persistance ou de réduction du contexte (Memory Management) avez-vous mise en place pour éviter l'explosion des coûts en tokens et la perte d'informations critiques entre le premier et le dernier agent du flux ? (Nous évaluons ici la gestion du contexte window et des tokens).